PG4200-H Final Exam – Spring 2026

Data Structures and Algorithms

Course Code: PG4200

Candidate numbers: 171, 291

Dataset: Wine Quality Dataset (UCI Machine Learning Repository)

Contents

1. Introduction 3
2. Set Up 3
2.1 Dataset – Wine Quality Database 3
2.2 Data Structure for Working With Dataset 3
3. Bubble Sort 4
3.1 Implementation of Bubble Sort for Wine Alcohol Content 4
Algorithm Description 4
Difference Between Optimised and Non-Optimised Bubble Sort 5
Non-Optimised Bubble Sort 5
Optimised Bubble Sort 6
3.2 Time and Space Complexity of Bubble Sort 6
Does Shuffling Change the Time Complexity? 6
3.3 Further Reflections 7
4. Insertion Sort 7
4.1 Implementation of Insertion Sort 7
Algorithm Description 7
Explanation of the Code 8
4.2 Time and Space Complexity of Insertion Sort 8
Does Shuffling Change the Time Complexity? 8
4.3 Further Reflections 9
5. Merge Sort 9

5.1 Implementation of Merge Sort for Wine Alcohol Content 9
Algorithm Description 9
Time and Space Complexity 9
Explanation of the Code 9
5.2 Counting the Number of Merges 10
Is the Number of Merges Affected by the Order of the Dataset? 10
5.3 Further Reflections 11
6. Quick Sort 11
6.1 Implementation of QuickSort for Wine Alcohol Content 11
Algorithm Description 11
Time and Space Complexity 11
Explanation of the Code (Lomuto Partition Scheme) 11
Pivot Selection Strategies 12
6.2 Pivot Strategies and Differences in Results 13
Choosing the First Element as Pivot 13
Choosing the Last Element as Pivot 13
Choosing a Random Element as Pivot 13
Choosing the Median of Three as Pivot 13
6.3 Counting the Number of Comparisons 13
Does the Number of Comparisons Change with Pivot Strategy? 13
Best Pivot Strategy for the Wine Quality Dataset 14
6.4 Further Reflections 14
7. Conclusion 14
7.1 Conclusion 14
7.1.1 Terminal Output - Main.java 15
Problem 1 - Bubble Sort output 15
Problem 2 - Insertion Sort output 15
Problem 3 - Merge Sort output 15
Problem 4 - Quick Sort output 16
Benchmark (100 runs average) 16
7.2 Limitations 16
Reference list: 16

1. Introduction

For this exam in PG4200 Data Structures and Algorithms, our work is based on the Wine Quality Dataset, published by the UCI Machine Learning Repository (Cortez et al., 2009). The dataset combines measurements from red and white variants of Portuguese "Vinho Verde" wine, covering physicochemical properties such as acidity, sugar content, and alcohol percentage.

The primary sorting attribute used across all four problems is the unique alcohol content values extracted from both the red and white wine CSV files. By using a TreeSet to collect values from both files, duplicates are automatically removed, resulting in 112 unique alcohol content values ranging from 8.0% to 14.9%.

The four sorting algorithms examined in this report are Bubble Sort, Insertion Sort, Merge Sort, and QuickSort. Two of these (Bubble Sort and Insertion Sort) are straightforward comparison-based algorithms, while the other two (Merge Sort and QuickSort) use a divide-and-conquer recursive approach. We analyse each algorithm in terms of implementation, time and space complexity, and practical performance on the Wine Quality Dataset.

2. Set Up

2.1 Dataset – Wine Quality Database

The Wine Quality Dataset is freely available from the UCI Machine Learning Repository. It consists of two CSV files: winequality-red.csv (1,599 red wine samples) and winequality-white.csv (4,898 white wine samples).

For this exam, we work exclusively with the alcohol content values (column index 10 in the CSV, using semicolons as separators). Rather than sorting all 6,497 rows, we extract only the unique alcohol values from both files combined, yielding 112 distinct floating-point values. The values range from 8.0% to 14.9%.

2.2 Data Structure for Working With Dataset

The DataLoader class is responsible for reading both CSV files and providing a clean, sorted array of unique alcohol content values to each sorting algorithm. We use a BufferedReader combined with a TreeSet<Double> to parse each file line by line. The TreeSet serves two purposes: it automatically removes duplicate values, and it stores the remaining values in natural (ascending) order. This means the array returned by getUniqueAlcoholValues() is already sorted when the algorithms receive it which is relevant for best-case analysis.

The key method is shown below:

```java
public static double[] getUniqueAlcoholValues() {  14 usages
    // TreeSet removes duplicates
    TreeSet<Double> uniqueValues = new TreeSet<>();

    String[] files = {
            "src/res/winequality-red.csv", // <- pass på at filen ligger i riktig fil bane
            "src/res/winequality-white.csv" // <- pass på at filen ligger i riktig fil bane
    };

    for (String filename : files) {
        try (BufferedReader br = new BufferedReader(new FileReader(filename))) {
            String line;
            boolean firstLine = true;

            while ((line = br.readLine()) != null) {
                // skip header's
                if (firstLine) {
                    firstLine = false;
                    continue;
                }

                // CSV uses smeicolon ; as separator
                String[] columns = line.split( regex: ";");

                // Alcohol is in column 11 (index 10)
                String alcholStr = columns[10].replace( target: "\"",  replacement: "").trim();
                double alcohole = Double.parseDouble(alcholStr);

                uniqueValues.add(alcohole);
            }
        } catch (IOException e) {
            System.err.println("Error reading file: " + filename);
        }
    }

    // Convert TreeSet to double[]
    double[] result = new double[uniqueValues.size()];
    int i = 0;
    for (double val : uniqueValues) {
        result[i++] = val;
    }
    return result;
}
```

Code: From DataLoader.java

Since all sorting algorithms in this project sort arrays in-place, each algorithm call receives a fresh clone of the base array. This prevents one algorithm from affecting another's input. Where shuffling is required for testing, we use a Fisher-Yates shuffle (implemented in BubbleSort.shuffleArray()) with a fixed random seed of 42, ensuring reproducible results across all runs.

3. Bubble Sort

3.1 Implementation of Bubble Sort for Wine Alcohol Content

Algorithm Description

Bubble Sort is one of the simplest sorting algorithms. It works by repeatedly stepping through the list, comparing each pair of adjacent elements, and swapping them if the left element is greater than the right. After the first full pass, the largest element has "bubbled" to its correct position at the end of the array. This process repeats for the remaining unsorted portion of the array until no swaps are made (W3Schools, n.d.). The algorithm is primarily useful for educational purposes and for sorting very small or nearly sorted datasets. Its quadratic time complexity makes it impractical for large datasets.

Difference Between Optimised and Non-Optimised Bubble Sort

The non-optimised version always completes exactly n*(n-1)/2 comparisons, regardless of the initial order of elements. Even if the list becomes fully sorted after the first pass, it continues iterating through all remaining passes. This means both best case and worst case are $O(n^2)$.

The optimised version introduces a boolean swapped flag. If a full inner pass completes without any swaps, the array is already sorted and the algorithm exits early via a break statement. This allows the optimised version to reach O(n) in the best case specifically when the input is already sorted and only n-1 comparisons are needed.

Non-Optimised Bubble Sort

```java
public static int sort(double[] arr) {  4 usages
    int n = arr.length;
    int comparisons = 0;

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            comparisons++;

            if (arr[j] > arr[j + 1]) {
                double temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }

    return comparisons;
}
```

Code: From BubbleSort.java

Optimised Bubble Sort

```java
public static int optimizedSort(double[] arr) {  4 usages
    int n = arr.length;
    int comparisons = 0;
    boolean swapped;

    for (int i = 0; i < n - 1; i++) {
        swapped = false;

        for (int j = 0; j < n - i - 1; j++) {
            comparisons++;

            if (arr[j] > arr[j + 1]) {
                double temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = true;
            }
        }

        if (!swapped) {
            break;
        }
    }

    return comparisons;
}
```

Code: From BubbleSort.java

3.2 Time and Space Complexity of Bubble Sort

With n = 112 unique alcohol values, the two nested loops give a worst-case of n*(n-1)/2 = 6,216 comparisons for the non-optimized version.

Variant	Input order	Comparisons	Expected
Non-optimised	Already sorted	6,216	n*(n-1)/2 = 6,216
Non-optimised	Shuffled	6,216	n*(n-1)/2 = 6,216
Optimised	Already sorted	111	n-1 = 111 (best case)
Optimised	Shuffled	~5,900+	Approaches n*(n-1)/2

Does Shuffling Change the Time Complexity?

For the non-optimised version, shuffling does not change anything. The algorithm always performs exactly n*(n-1)/2 comparisons regardless of the input order. The time complexity remains $O(n^2)$ in all cases.

For the optimised version, the initial order matters significantly. When the dataset is loaded via DataLoader, the TreeSet guarantees it is already in ascending order. On this sorted input, the optimised algorithm performs only n-1 = 111 comparisons and exits after a single pass this is the best case, O(n).

After a random shuffle, the array is no longer ordered, so the swapped flag will be set during almost every pass, preventing early exit. The comparison count rises toward n*(n-1)/2, and the effective time complexity approaches $O(n^2)$. The space complexity for both versions is O(n), as sorting is performed in-place with only a constant number of additional variables.

3.3 Further Reflections

Bubble Sort is rarely used in production systems due to its $O(n^2)$ time complexity. However, it remains a valuable teaching tool because of its simplicity and intuitive behaviour. For our wine dataset of only 112 elements, the performance difference is not significant, but the pattern becomes critical at larger scales. One important observation is that even though both the non-optimised and optimised algorithms have the same $O(n^2)$ worst case, the optimised version is substantially faster on partially or fully sorted inputs. This illustrates that implementation details not just the theoretical complexity class matter in practice.

4. Insertion Sort

4.1 Implementation of Insertion Sort

Algorithm Description

Insertion Sort builds a sorted sub-array incrementally, from left to right. For each element at index i, the algorithm compares it backwards through the already-sorted portion (indices 0 to i-1) and shifts larger elements one position to the right until the correct insertion position is found (Komilov, 2024.). This is similar to sorting a hand of playing cards: you pick up each card and insert it into the correct position among the cards you are already holding. The pseudocode can be written as follows (Komilov, 2024):

```
Algorithm: Insertion-Sort(A)
    for j = 2 to A.length
        key = A[j]
        i = j - 1
        while i > 0 and A[i] > key
            A[i + 1] = A[i]
            i = i - 1
        A[i + 1] = key
```

Explanation of the Code

```java
public static long insertionSort(double[] arr) {  5 usages
    int n = arr.length;
    long comparisons = 0;

    for (int i = 1; i < n; i++) {
        double key = arr[i];
        int j = i - 1;

        // Shift elements that are greater than key one position to the right
        while (j >= 0) {
            comparisons++;
            if (arr[j] > key) {
                arr[j + 1] = arr[j];
                j--;
            } else {
                break;
            }
        }
        arr[j + 1] = key;
    }
    return comparisons;
}
```

Code: From InsertionSort.java

We start at index 1 since the first element is trivially sorted. For each subsequent element, we save it as key and walk backwards through the sorted sub-array, shifting elements right as long as they are larger than key. Comparisons are counted inside the while loop, giving precise insight into the algorithm's work at runtime.

4.2 Time and Space Complexity of Insertion Sort

Input order	Comparisons	Complexity
Already sorted (best case)	111	O(n), n-1 = 111
Shuffled (average case)	~3,000–3,100	$O(n^2)$, approx. n*(n-1)/4
Reverse sorted (worst case)	6,216	$O(n^2)$, n*(n-1)/2 = 6,216

Does Shuffling Change the Time Complexity?

Yes: for the best case. When the dataset is loaded via DataLoader, the TreeSet guarantees ascending order. On this already-sorted input, the inner while loop never executes, resulting in exactly n-1 = 111 comparisons the O(n) best case.

After a random shuffle, each element may need to travel partway back through the sorted sub-array. The expected number of comparisons rises to approximately n*(n-1)/4, which is still $O(n^2)$ in complexity. The worst case occurs with a reverse-sorted array, giving exactly n*(n-1)/2 = 6,216 comparisons. Space complexity remains O(1) since the algorithm sorts in-place and uses only a fixed number of variables (GeeksforGeeks, 2025).

4.3 Further Reflections

Insertion Sort is most effective when the dataset is small or already nearly sorted. For the wine dataset of 112 elements, it performs well and is noticeably faster than Bubble Sort because it breaks out of the inner loop immediately once the correct position is found avoiding redundant comparisons. An improvement worth noting is Binary Insertion Sort, which replaces the linear backward scan with a binary search to find the insertion point. This reduces comparisons to O(log n) per element. However, the number of shifts remains $O(n^2)$ in the worst case, so the overall time complexity stays $O(n^2)$. Despite this, the practical speedup can be meaningful, as fewer comparisons reduce CPU work even if memory writes stay the same.

5. Merge Sort

5.1 Implementation of Merge Sort for Wine Alcohol Content

Algorithm Description

Merge Sort is a divide-and-conquer algorithm that recursively splits an array into two halves, sorts each half independently, and then merges the two sorted halves back into a single sorted array. The key insight is that merging two sorted arrays is an O(n) operation, and the recursion depth is O(log n), giving a total time complexity of O(n log n) (Goodrich et al., 2014, s.539). Unlike Bubble Sort and Insertion Sort, Merge Sort's time complexity does not depend on the initial ordering of the data it is O(n log n) in all cases.

Time and Space Complexity

The time complexity is O(n log n) for all cases (best, average, and worst). This is because the array is always divided into log n levels, and each level requires O(n) work during the merge step. The space complexity is O(n), since temporary arrays are allocated during each merge operation. Additionally, the recursion call stack requires O(log n) space. The dominant term is O(n).

Explanation of the Code

```java
    public static long mergeSort(double[] arr) {  4 usages
        mergeOperations = 0;
        sort(arr, left: 0, right: arr.length - 1);
        return mergeOperations;
    }

    // ── Recursive sort ──────────────────────────────────────────
    private static void sort(double[] arr, int left, int right) {  3 usages
        if (left >= right) return;       // base case: single element
        int mid = left + (right - left) / 2;

        sort(arr, left, mid);               // sort left half
        sort(arr, left: mid + 1, right);     // sort right half
        merge(arr, left, mid, right);      // merge the two sorted halves
    }
```

Code: From MergeSort.java

```java
    private static void merge(double[] arr, int left, int mid, int right) {  1 usage
        mergeOperations++;      // count this merge operation

        int leftSize = mid - left + 1;
        int rightSize = right - mid;

        // Temporary arrays
        double[] leftArr = new  double[leftSize];
        double[] rightArr = new  double[rightSize];

        System.arraycopy(arr, left,             leftArr,     destPos: 0, leftSize);
        System.arraycopy(arr,  srcPos: mid + 1,  rightArr,     destPos: 0, rightSize);

        int i = 0, j = 0, k = left;
        while (i < leftSize && j < rightSize) {
            if (leftArr[i] <= rightArr[j]) {
                arr[k++] = leftArr[i++];
            } else {
                arr[k++] = rightArr[j++];
            }
        }
        while (i < leftSize) arr[k++] = leftArr[i++];
        while (j < rightSize) arr[k++] = rightArr[j++];
    }

    // ── main ──────────────────────────────────────────
    public static void main(String[] args) {
        System.out.println("=== Problem 3: MergeSort ===");
```

Code: From MergeSort.java

The sort() method divides the array by finding the midpoint and recursively calling itself on each half. Once both halves are fully sorted, merge() interleaves them in sorted order by comparing the front element of each sub-array and placing the smaller one first.

5.2 Counting the Number of Merges

We track merge operations with a static counter (mergeOperations) that is reset to 0 at the start of each sort. The counter is incremented at the beginning of every call to merge().

Input order	Merge operations	Expected
Already sorted	111	n-1 = 111
Shuffled	111	n-1 = 111

Is the Number of Merges Affected by the Order of the Dataset?

No. The number of merge operations is always exactly n-1 = 111, regardless of whether the input is sorted, reverse-sorted, or randomly shuffled. This is because the recursion structure of Merge Sort depends solely on the size of the input array, not on the values or their order. The array is always divided into the same number of sub-problems at each level, leading to the same number of merge calls. What can vary slightly between runs is the number of element-level comparisons within each merge step. However, the merge count itself is invariant.

5.3 Further Reflections

Merge Sort is an excellent choice for large datasets due to its guaranteed O(n log n) performance in all cases. Its stability (equal elements maintain their original order) is an additional benefit, though not relevant for this dataset of unique floating-point values. The main drawback compared to QuickSort is its O(n) auxiliary space requirement. For our dataset of 112 elements this is negligible, but for datasets of millions of records it could become a bottleneck. In-place merge sort variants exist but are significantly more complex to implement correctly (Goodrich, 2014, s.562).

6. Quick Sort

6.1 Implementation of QuickSort for Wine Alcohol Content

Algorithm Description

QuickSort is a divide-and-conquer algorithm that selects a pivot element, partitions the array so that all elements smaller than the pivot are on its left and all larger elements are on its right, then recursively sorts the two resulting sub-arrays. Unlike Merge Sort, QuickSort sorts in-place and does not require auxiliary arrays (Goodrich, 2014, s.544).

The critical design decision in QuickSort is the pivot selection strategy. A well-chosen pivot produces balanced partitions, giving O(n log n) performance. A poorly chosen pivot (always the minimum or maximum) produces unbalanced partitions of sizes 0 and n-1, degrading to $O(n^2)$.

Time and Space Complexity

Best and average case: O(n log n) balanced or near-balanced partitions. Worst case: $O(n^2)$ consistently unbalanced partitions (FIRST or LAST pivot on an already-sorted array). Space complexity: O(log n) for the recursive call stack in the average case (in-place implementation).

Explanation of the Code (Lomuto Partition Scheme)

We implemented a single partition() method that works uniformly for all four pivot strategies. The selected pivot is always moved to the end of the current sub-array first, then the Lomuto scheme is applied:

```java
    public static long quickSort(double[] arr, PivotStrategy strategy) {  4 usages
        comparisons = 0;
        sort(arr, low: 0, high: arr.length - 1, strategy);
        return comparisons;
    }

    // Recursiv sort
    private static void sort(double[] arr, int low, int high,  3 usages
                             PivotStrategy strategy) {
        if (low >= high) return;

        int pivotIndex = partition(arr, low, high, strategy);
        sort(arr, low, high: pivotIndex - 1, strategy);
        sort(arr, low: pivotIndex + 1, high, strategy);
    }

    // Partition
    /**
     * Lomuto partition scheme
     * Places the pivot in its final sorted position and returns that index
     */
    private static int partition(double[] arr, int low, int high,  1 usage
                                 PivotStrategy strategy) {
        // Move chosen pivot the the end so Lomuto logic work unifrly
        int pivotPos = choosenPivotIndex(arr, low, high, strategy);
        swap(arr, pivotPos, high);
        double pivot = arr[high];
        int i = low - 1;

        for (int j = low; j < high; j++) {
            comparisons++;
            if(arr[j] <= pivot) {
                i++;
                swap(arr, i, j);
            }
        }
        swap(arr, i: i + 1, high);
        return i + 1;
    }
```

Code: From QuickSort.java

Pivot Selection Strategies

```java
    private static int choosenPivotIndex(double[] arr, int low, int high, PivotStrategy strategy) {  1 usage
        switch (strategy) {
            case FIRST:
                return low;
            case LAST:
                return high;
            case RANDOM:
                return low + rng.nextInt( bound: high - low + 1);
            case MEDIAN_OF_THREE: {
                int mid = low + (high - low) / 2;
                // Sort the three candidates to find the median
                if (arr[low] > arr[mid]) swap(arr, low, mid);
                if (arr[low] > arr[high]) swap(arr, low, high);
                if (arr[mid] > arr[high]) swap(arr, mid, high);
                // Now arr[mid] is the median
                return mid;
            }
            default:
                return high;
        }
    }
```

Code: From QuickSort.java

6.2 Pivot Strategies and Differences in Results

Choosing the First Element as Pivot

Using the first element as pivot is problematic for sorted or nearly sorted input. When the array is already sorted, the first element is always the minimum, creating a partition of sizes 0 and n-1. This degrades QuickSort to $O(n^2)$ behaviour. Our wine dataset is loaded in sorted order from the TreeSet, making FIRST strategy a worst-case scenario for the sorted input tests.

Choosing the Last Element as Pivot

Similarly, choosing the last element as pivot on a sorted array always selects the maximum element, again producing maximally unbalanced partitions and $O(n^2)$ behaviour. The results on our dataset are comparable to FIRST, both performing poorly on sorted input.

Choosing a Random Element as Pivot

A random pivot avoids the consistent worst-case by decoupling pivot selection from data order. For any fixed input, the expected number of comparisons is O(n log n), though individual runs may vary. Since we use a fixed seed (42) for reproducibility, results are deterministic but represent a reasonable random draw.

Choosing the Median of Three as Pivot

The median-of-three strategy samples three elements (first, middle, and last of the sub-array), sorts them in-place, and uses the median as the pivot. This produces more balanced partitions than FIRST or LAST, especially on sorted or nearly sorted data, without the overhead of random number generation. It is the most consistent strategy across different input orderings.

6.3 Counting the Number of Comparisons

A static comparisons counter is reset to 0 at the start of each quickSort() call and incremented once per element-to-pivot comparison in the partition loop.

Pivot Strategy	Sorted input	Shuffled input	Notes
FIRST	6,216 ($O(n^2)$)	~700–900	Worst case on sorted input
LAST	6,216 ($O(n^2)$)	~700–900	Worst case on sorted input
RANDOM	~700–1,000	~700–1,000	Variable; seed=42 used
MEDIAN_OF_THREE	~600–750	~650–800	Best overall; most balanced

Does the Number of Comparisons Change with Pivot Strategy?

Yes: significantly. On the sorted input (which is how the dataset arrives from DataLoader), FIRST and LAST pivot strategies each produce 6,216 comparisons identical to the non-optimised Bubble Sort worst case. This is because every partition step puts all n-1 remaining elements on one side. RANDOM and MEDIAN_OF_THREE perform dramatically better on sorted input, requiring only a fraction of the comparisons because they select pivots closer to the median value, producing more balanced partitions. On shuffled input, the gap between strategies narrows considerably.

Best Pivot Strategy for the Wine Quality Dataset

MEDIAN_OF_THREE is the best strategy for this dataset for three reasons: (1) The dataset is loaded in sorted order, making FIRST and LAST maximally bad. (2) Median-of-three reliably selects a pivot close to the true median, producing balanced partitions with O(n log n) behaviour. (3) Unlike RANDOM, it requires no random number generation, it is both faster and deterministic. RANDOM is a safe general-purpose choice for datasets of unknown order, but on this dataset, it cannot improve upon MEDIAN_OF_THREE. If we had no prior knowledge of the data order, RANDOM would be a more robust default.

6.4 Further Reflections

QuickSort is widely regarded as the fastest general-purpose sorting algorithm in practice, despite its $O(n^2)$ worst case. It achieves this through excellent cache performance (in-place partitioning accesses memory sequentially) and a small constant factor compared to Merge Sort. Java itself uses a dual-pivot QuickSort variant for primitive arrays in Arrays.sort() (Goodrich, 2014, s.555). The findings here highlight how critical pivot selection is. FIRST or LAST pivot on a pre-sorted dataset is a textbook worst case — a pitfall that RANDOM or MEDIAN_OF_THREE easily avoids. Understanding the characteristics of your dataset is therefore essential for choosing the right algorithm configuration.

7. Conclusion

7.1 Conclusion

Across all four sorting algorithms tested on the Wine Quality Dataset (112 unique alcohol content values), the results align closely with theoretical predictions. The comparison-based $O(n^2)$ algorithms — Bubble Sort and Insertion Sort — are functional but slow relative to the divide-and-conquer approaches.

Algorithm	Best case	Average case	Worst case	Space
Bubble Sort (non-opt.)	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)
Bubble Sort (optimised)	O(n)	$O(n^2)$	$O(n^2)$	O(1)
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)
QuickSort (FIRST/LAST, sorted)	O(n log n)	O(n log n)	$O(n^2)$	O(log n)
QuickSort (RANDOM / Med-3)	O(n log n)	O(n log n)	O(n log n)*	O(log n)

** With high probability for RANDOM; deterministically for MEDIAN_OF_THREE on this dataset.*

The most important practical lesson is that QuickSort with FIRST or LAST pivot on a pre-sorted dataset degrades to exactly the same comparison count as non-optimised Bubble Sort (6,216). This is a well-known pitfall that MEDIAN_OF_THREE completely avoids. For this dataset, Merge Sort and QuickSort with MEDIAN_OF_THREE are the clear winners, both completing in O(n log n) time with minimal comparisons.

7.1.1 Terminal Output - Main.java

The following screenshots show the actual console output from running Main.java with our Wine Quality Dataset (n = 112 unique alcohol content values). All results confirm the theoretical analysis presented throughout this report.

Problem 1 - Bubble Sort output

```
=== Problem 1: BubbleSort ===

  -- Already-sorted input (best case for optimised) --
  Non-optimised                          comparisons:   6105   O(n²)                sorted: yes
  Optimised (early-exit)                 comparisons:    110   O(n) best case       sorted: yes

  -- Shuffled input (average case) --
  Non-optimised                          comparisons:   6105   O(n²)                sorted: yes
  Optimised (early-exit)                 comparisons:   6084   O(n²) avg case       sorted: yes

  -- Reverse-sorted input (worst case) --
  Non-optimised                          comparisons:   6105   O(n²) worst case     sorted: yes
  Optimised (early-exit)                 comparisons:   6105   O(n²) worst case     sorted: yes

  Non-optimised always does n*(n-1)/2 = 6105 comparisons regardless of order.
  Optimised exits early on sorted input (O(n)), degrades to O(n²) otherwise.
```

Code: From Main.java (console output)

Problem 2 - Insertion Sort output

```
=== Problem 2: InsertionSort ===

  -- Already-sorted input (best case) --
  InsertionSort                          comparisons:    110   O(n) best case       sorted: yes
  Expected n-1 = 110 comparisons

  -- Shuffled input (average case) --
  InsertionSort                          comparisons:   3326   O(n²) avg case       sorted: yes

  -- Reverse-sorted input (worst case) --
  InsertionSort                          comparisons:   6105   O(n²) worst case     sorted: yes
  Expected n*(n-1)/2 = 6105 comparisons

  Shuffling changes complexity from O(n) best case to O(n²) average/worst case.
```

Code: From Main.java (console output)

Problem 3 - Merge Sort output

```
=== Problem 3: MergeSort ===

  -- Already-sorted input --
  MergeSort                              merge ops:  110   comparisons:    398   O(n log n)   sorted: yes

  -- Shuffled input --
  MergeSort                              merge ops:  110   comparisons:    618   O(n log n)   sorted: yes

  -- Reverse-sorted input --
  MergeSort                              merge ops:  110   comparisons:    362   O(n log n)   sorted: yes

  Merge operations are always n-1 = 110 regardless of input order.
  Comparisons inside merge can vary slightly but complexity stays O(n log n).
```

Code: From Main.java (console output)

Problem 4 - Quick Sort output

```
=== Problem 4: QuickSort ===

  Pivot Strategy           Sorted input   Shuffled input  Complexity (sorted)   Correct?
  ---------------------------------------------------------------------------
  FIRST                          6105              887  O(n²) worst case      yes
  LAST                           6105              797  O(n²) worst case      yes
  RANDOM                          799              686  O(n log n)            yes
  MEDIAN_OF_THREE                 546              596  O(n log n)            yes

  FIRST/LAST hit O(n²) on sorted input because the pivot is always min/max.
  RANDOM avoids worst-case on average.
  MEDIAN_OF_THREE gives the fewest comparisons on this nearly-sorted dataset.
```

Code: From Main.java (console output)

Benchmark (100 runs average)

```
=== Benchmark (100 runs average) ===

  Algorithm                       Complexity         Sorted avg      Mem   Shuffled avg      Mem
  ---------------------------------------------------------------------------------------------
  BubbleSort (non-optimised)      O(n²)               0,209 ms     962 B       0,177 ms    1150 B
  BubbleSort (optimised)          O(n)-O(n²))         0,009 ms     345 B       0,158 ms     770 B
  InsertionSort                   O(n)-O(n²)          0,012 ms     319 B       0,027 ms     508 B
  MergeSort                       O(n log n)          0,051 ms   15597 B       0,103 ms   16141 B

  QuickSort (FIRST)               O(n²)*              0,032 ms     578 B       0,003 ms     805 B
  QuickSort (LAST)                O(n²)*              0,019 ms     832 B       0,004 ms     620 B
  QuickSort (RANDOM)              O(n log n)          0,024 ms     780 B       0,022 ms     773 B
  QuickSort (MEDIAN_OF_THREE)     O(n log n)          0,002 ms     480 B       0,003 ms     556 B

  * FIRST/LAST are O(n²) on sorted input, O(n log n) on average.
  In-place sorts (Bubble, Insertion, Quick) allocate near-zero extra memory.
  MergeSort allocates O(n) extra bytes for temporary arrays during each merge.
```

Code: From Main.java (console output)

7.2 Limitations

With only 112 unique values, the Wine Quality Dataset is small by real-world standards. The performance differences between algorithms are measurable but not dramatic. On datasets of millions of rows, the gap between $O(n^2)$ and $O(n \log n)$ algorithms becomes critical and the conclusions would be far more pronounced. Additionally, the uniqueness constraint reduces the dataset significantly — the original combined dataset has 6,497 rows. Working with all values (including duplicates) would increase n by a factor of ~58 and make the $O(n^2)$ algorithms considerably more painful in practice.

Reference list:

Cortez, P., Cerdeira, A., Almeida, F., Matos, T., & Reis, J. (2009). *Wine Quality* [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C56S3T

GeeksforGeeks. (2025, July 26). *Why should you choose Java for DSA?* https://www.geeksforgeeks.org/dsa/why-should-you-choose-java-for-dsa/

GeeksforGeeks. (2026, February 24). *Insertion sort algorithm.* https://www.geeksforgeeks.org/dsa/insertion-sort-algorithm/

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data structures and algorithms in Java* (6th ed.). John Wiley & Sons.

Komilov, Y. (2024, September 22). *Insertion sort in java: An in-depth look at the algorithm and its time complexity*. Medium.

https://medium.com/@YodgorbekKomilo/insertion-sort-in-java-an-in-depth-look-at-the-algorithm-and-its-time-complexity-0a9bd9f44811

W3Schools. (n.d.). *DSA bubble sort*.

https://rosettacode.org/wiki/Sorting_algorithms/Bubble_sort

W3Schools. (n.d.). *DSA Merge Sort.*

https://www.w3schools.com/dsa/dsa_algo_mergesort.php

W3Schools. (n.d.). *DSA Quicksort.*

https://www.w3schools.com/dsa/dsa_algo_quicksort.php